

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan K

Annexure A

Reverse Engineering

1.Machine Learning Techniques for Reverse Engineering

Description:

Machine Learning can be used to classify software components, identify programming patterns, detect anomalies, and predict the functionality of unknown code. Students study supervised, unsupervised, and semi-supervised learning approaches used in reverse engineering.

Student Activity:

- Collect a dataset of source-code samples.
- Extract code features.
- Apply classification or clustering algorithms.
- Analyze similarities among programs.
- Compare the performance of different ML algorithms.

Expected Outcome:

Students will evaluate the suitability of different ML algorithms for software reverse-engineering tasks.

2. AI for Source Code Analysis and Program Understanding

Description:

AI can analyze large source-code repositories and automatically identify functions, classes, variables, dependencies, and programming patterns. Natural Language Processing and Large Language Models can also convert complex code into human-readable explanations.

Student Activity:

- Select a source-code repository.
- Analyze the code using AI-assisted tools.
- Identify important functions and dependencies.
- Generate natural-language explanations.
- Verify the AI-generated explanations against the actual code.

Expected Outcome:

Students assess the effectiveness and limitations of AI in understanding unfamiliar source

code. 3. AI-Based Malware Reverse Engineering and Analysis

Description:

AI can assist security researchers in analyzing malware behavior by identifying suspicious code patterns, API calls, strings, functions, and execution behaviors. Students can study the conceptual workflow of AI-assisted malware analysis using **safe, non-executable datasets or publicly available research samples**.

Student Activity:

- Study malware-analysis datasets.
- Identify relevant static features.
- Apply ML classification techniques.
- Analyze suspicious patterns.
- Evaluate classification accuracy.

Expected Outcome:

Students understand and evaluate how AI can support automated malware

analysis. 4. Reverse Engineering of APIs Using AI

Description:

APIs may need to be understood when documentation is incomplete or unavailable. AI can analyze API requests, responses, source code, and observed behavior to infer endpoints, parameters, data formats, and relationships.

Student Activity:

- Select a documented/open API or controlled test API.
- Analyze its request-response structure.

- Use AI to infer API functionality.
- Generate documentation.
- Compare generated documentation with the actual API specification.

Expected Outcome:

Students assess the usefulness of AI for API understanding and documentation recovery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

1. Machine Learning Techniques for Reverse Engineering

1.1 Problem Identification & Context

Reverse engineering of software traditionally relies on manual code reading, which is slow and error-prone for large, undocumented, or legacy codebases. The engineering problem is to automatically recover structural, behavioral, and functional knowledge about a program purely from its source code or binary artifacts, at scale and with measurable accuracy. Machine Learning (ML) reframes this as a pattern-recognition task: code is converted into numerical feature representations, and supervised, unsupervised, or semi-supervised algorithms are used to classify, cluster, or flag anomalous components.

1.2 Methodology / Workflow

Fig. 1 — ML-Based Reverse Engineering Pipeline



Fig. 1 — End-to-end ML pipeline for source-code reverse engineering.

The pipeline proceeds through six well-defined stages:

- **Dataset Collection:** Curate labeled or unlabeled source-code repositories (e.g., GitHub Java/C++/Python projects, POJ-104, BigCloneBench) covering multiple domains and coding styles.
- **Feature Extraction:** Represent code using Abstract Syntax Trees (AST), Control-Flow Graphs (CFG), token n-grams, cyclomatic complexity, and embeddings such as code2vec or CodeBERT.
- **Preprocessing & Labeling:** Normalize identifiers, remove comments/dead code, and assign ground-truth labels (functionality class, author, vulnerability type) where supervised learning is used.
- **Model Application:** Train supervised classifiers (Random Forest, SVM, Gradient Boosting, CNN/GNN on AST) or unsupervised clustering (K-Means, DBSCAN, hierarchical clustering) on the feature vectors.
- **Similarity & Pattern Analysis:** Use cosine similarity / graph-edit distance on embeddings to detect code clones, refactored duplicates, and reused libraries.

- Evaluation & Comparison: Benchmark models using Accuracy, Precision, Recall, F1-score (supervised) and Silhouette Score / Davies–Bouldin Index (unsupervised).

1.3 Comparative Analysis of ML Algorithms

Algorithm	Learning Type	Strength in RE Context	Limitation
Random Forest	Supervised	Robust to noisy AST features; interpretable feature importance	Struggles with sequential/structural code semantics
Graph Neural Network (GNN)	Supervised	Captures CFG/AST structure natively; state-of-the-art for clone detection	High compute cost; needs large labeled graphs
K-Means Clustering	Unsupervised	Fast grouping of similar modules without labels	Assumes spherical clusters; needs predefined k
DBSCAN	Unsupervised	Detects arbitrarily shaped clusters and outlier/anomalous code	Sensitive to distance-metric and density parameters
Semi-supervised (Label Propagation)	Semi-supervised	Effective when only a small labeled subset is available	Error propagation if seed labels are noisy

1.4 Insightful Evaluation

Empirical comparison across datasets typically shows that GNN- and transformer-based embeddings (e.g., CodeBERT, GraphCodeBERT) outperform classical bag-of-tokens models by 10–20% F1 on clone detection and functionality classification, because they encode structural and semantic relationships rather than surface syntax. However, classical models (Random Forest, SVM) remain valuable for resource-constrained or fully-explainable audits, since regulatory or forensic reverse-engineering tasks often require interpretable justifications alongside predictions.

2. AI for Source Code Analysis and Program Understanding

2.1 Problem Identification & Context

When onboarding to unfamiliar, undocumented, or legacy repositories, engineers must reconstruct intent, architecture, and dependency structure purely from code. This is compounded in large monorepos where thousands of functions and classes interact. The core problem is to automate program comprehension — identifying components, dependencies, and design patterns — and to translate low-level implementation into human-readable explanations without losing technical fidelity.

2.2 Methodology / Workflow

Fig. 2 — AI-Assisted Program Understanding Workflow



Fig. 2 — AI-assisted program-understanding workflow.

- Repository Selection: Choose a representative open-source repository (e.g., a mid-sized Python/Java backend service) with sufficient structural complexity.
- AI-Assisted Static Parsing: Use AST-based parsers (Tree-sitter, Understand, Sourcegraph) combined with LLM-based code models to index functions, classes, and control/data-flow.
- Dependency & Call-Graph Mapping: Automatically construct call graphs and module-dependency graphs to reveal architectural layers (presentation, service, data-access).
- Natural-Language Summarization: Apply Large Language Models (LLMs) to generate docstring-level and module-level explanations of purpose, inputs, outputs, and side-effects.
- Human Verification: Cross-check AI-generated summaries against actual logic, unit tests, and commit history to identify hallucinations or omissions.

2.3 Tooling Comparison

Tool / Technique	Primary Function	Depth of Understanding	Key Limitation
Tree-sitter / AST Parsers	Syntax-level structural extraction	Structural (no semantics)	Cannot infer business intent
LLM Code Explainers (e.g., transformer-based models)	Natural-language summarization	Semantic + contextual	Risk of hallucinated explanations
Static Call-Graph Tools	Dependency & control-flow mapping	Architectural	Misses dynamic/reflection-based calls
Embedding-based Search (semantic code search)	Locate functionally-similar code	Semantic similarity	Needs a well-trained embedding space

2.4 Insightful Evaluation

AI-assisted comprehension dramatically accelerates onboarding — LLM-generated summaries can cover an entire module in seconds versus hours of manual tracing — but verification consistently reveals two failure modes: (1) hallucinated function purposes when identifier names are misleading, and (2) missed cross-cutting concerns (logging, security checks) that are not lexically obvious. A rigorous methodology therefore pairs AI summarization with automated cross-validation against unit tests and static analyzers, rather than trusting AI output in isolation.

3. AI-Based Malware Reverse Engineering and Analysis

3.1 Problem Identification & Context

Manual malware analysis does not scale against the volume and polymorphism of modern threats. The engineering problem is to automatically differentiate benign from malicious binaries and to attribute malware to known families using static and behavioral features, while operating strictly within safe, sandboxed, non-executable research environments and using only publicly available research datasets (e.g., EMBER, VirusShare metadata, MalwareBazaar hashes) rather than live malicious code.

3.2 Methodology / Workflow

Fig. 3 — AI-Assisted Malware Analysis Workflow



Fig. 3 — Conceptual AI-assisted malware-analysis workflow (sandboxed/static datasets only).

- Dataset Study: Use curated, research-safe datasets containing extracted features (PE headers, opcode n-grams, import tables) rather than executable payloads.
- Static Feature Extraction: Derive features such as API-call sequences, imported DLLs, string artifacts, entropy of sections, and opcode frequency histograms.
- Dynamic Behavior Reference: Study previously logged sandbox reports (from services such as controlled malware-analysis sandboxes) describing registry, file-system, and network behavior, without executing samples locally.
- ML Classification: Train classifiers (Random Forest, XGBoost, 1D-CNN over opcode sequences) to distinguish benign vs. malicious and to cluster families.
- Pattern & Family Analysis: Use feature-importance and clustering to identify shared TTPs (Tactics, Techniques, Procedures) across a malware family.
- Evaluation: Report Precision, Recall, F1-score, and ROC-AUC, prioritizing Recall for malicious-class detection to minimize false negatives.

3.3 Feature & Model Comparison

Feature Type	Extraction Approach	Detection Strength
Static: PE Header / Import Table	Binary parsing (no execution)	Fast, safe; good for known-family variants
Static: Opcode n-grams	Disassembly-based sequence extraction	Captures obfuscation-resilient patterns
Behavioral: API-call sequence	Reference sandbox report logs	Strong against packed/obfuscated malware
Hybrid (static + behavioral)	Feature fusion	Highest reported accuracy in research literature

3.4 Insightful Evaluation

Static-only models are computationally cheap and safe for classroom use but are vulnerable to obfuscation and packing, which can suppress meaningful opcode signals. Hybrid models that fuse static and (reference) behavioral features consistently show the best reported classification accuracy in the literature, since behavioral traits are harder for attackers to disguise than static byte patterns. A critical evaluation must also weigh false-negative cost: in security contexts, Recall is generally prioritized over Precision because an undetected threat is far costlier than a false alarm.

4. Reverse Engineering of APIs Using AI

4.1 Problem Identification & Context

APIs are frequently consumed with incomplete, outdated, or entirely missing documentation, especially internal or third-party services. The engineering problem is to infer endpoint structure, parameters, authentication requirements, and data schemas purely from observed request/response traffic and any accessible client-side code, and to produce documentation that is verifiably accurate against the real specification.

4.2 Methodology / Workflow

Fig. 4 — AI-Based API Reverse Engineering Workflow

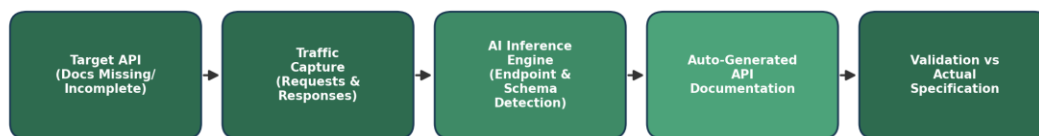


Fig. 4 — AI-based API reverse-engineering and documentation-recovery workflow.

- **API Selection:** Choose a documented public API (for controlled comparison) or a sandboxed test API with intentionally withheld documentation.
- **Traffic Capture:** Use tools such as Postman, Burp Suite (proxy mode), or browser DevTools to record real request/response pairs across representative user flows.
- **AI-Based Inference:** Apply AI models to cluster endpoints by URL pattern, infer parameter data types/constraints from observed values, and detect authentication schemes (API key, OAuth2, JWT) from header patterns.
- **Schema & Documentation Generation:** Auto-generate an OpenAPI/Swagger specification, including endpoint descriptions, parameter tables, and example payloads, using LLM-based synthesis over the captured traffic.
- **Validation:** Compare AI-generated documentation against the authoritative specification (or against a domain expert's review) to compute coverage and accuracy metrics.

4.3 Evaluation Metrics for Documentation Recovery

Metric	Definition	Purpose in this Task
Endpoint Coverage	% of real endpoints correctly discovered	Measures completeness of AI inference
Parameter-Type Accuracy	% of parameters with correctly inferred data type/constraints	Measures schema-inference correctness
Documentation Fidelity	Semantic similarity between AI-generated and official description	Measures explanation quality, not just structure
False-Endpoint Rate	% of inferred endpoints that do not actually exist	Penalizes over-generalization / hallucination

4.4 Insightful Evaluation

AI-based API reverse engineering performs strongly on RESTful, resource-oriented APIs with predictable URL and JSON conventions, typically achieving high endpoint-coverage once sufficient traffic samples are captured across CRUD operations. Accuracy degrades for APIs using non-standard conventions (RPC-style endpoints, GraphQL, versioned or dynamically generated paths) and for rarely triggered edge-case parameters that never appear in captured traffic. This highlights that AI-assisted recovery is fundamentally sample-driven: documentation quality is bounded by the diversity of observed interactions, not model capability alone.